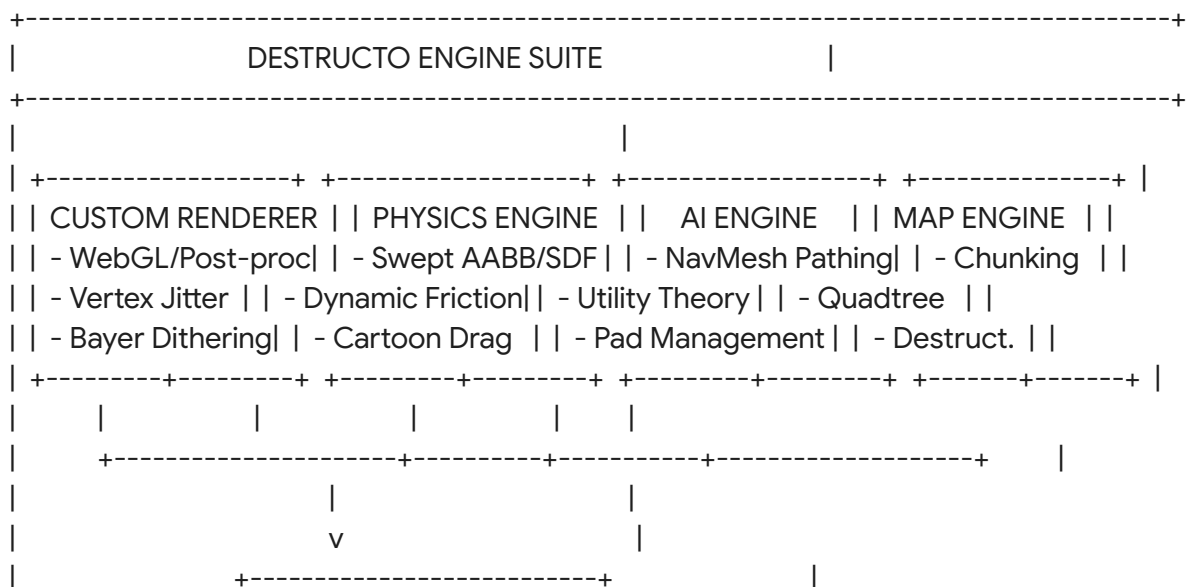
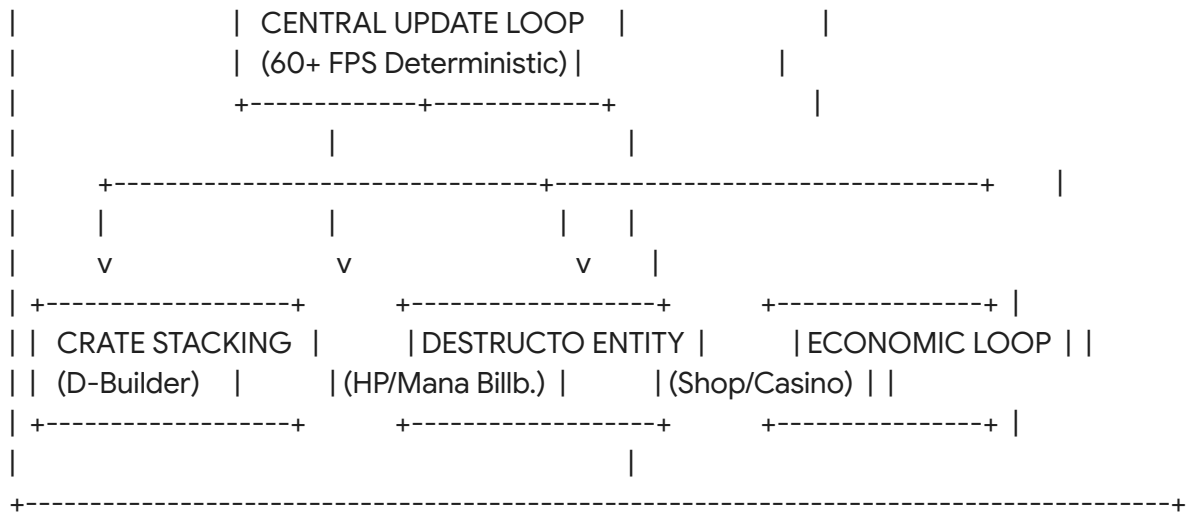


# Complete Architectural Specification for Destructo: A Modern High-Performance Web-Native Clone and Expansion of the Classic Team Buddies Tactical Framework

This document details the complete architectural and engineering specification for *Destructo*, a high-performance, web-native 3D action real-time strategy (A-RTS) game created by Aljay Leodones. Designed to perfectly replicate and expand upon the mechanical quirks, physics, and visual aesthetics of the PlayStation 1 classic *Team Buddies*, *Destructo* runs entirely within a modern browser context using Three.js and a suite of ten specialized third-party libraries.<sup>1</sup> To achieve an optimal gameplay experience, the architecture is decoupled into dedicated, custom-engineered subsystems designed to run at a deterministic 60+ frames per second (FPS).

## Core Architecture and Custom Subsystem Engines





## Custom WebGL Renderer Engine

The custom renderer utilizes `THREE.WebGLRenderer` configured with high-performance power preferences, zero anti-aliasing, and stencil buffers disabled to streamline drawing operations and minimize GPU state switches.<sup>6</sup> To capture the visual style of *Team Buddies* while leveraging modern graphics pipelines, *Destructo* implements a custom shader architecture consisting of two key stages<sup>5</sup>:

1. **Vertex-Snapping Jitter Shader:** Applied via the `onBeforeCompile` hook on all standard mesh materials. This shader projectively snaps vertex clip-space coordinates to a fixed low-resolution grid, replicating the lack of sub-pixel precision and fixed-point arithmetic of 32-bit retro consoles.
2. **Ordered Bayer Dithering Fragment Pass:** A post-processing filter that maps color outputs to a limited, retro-inspired color palette through a  $4 \times 4$  Bayer matrix.<sup>7</sup> This maps smooth lighting gradients into stylized pixel patterns reminiscent of classic software rasterization.<sup>7</sup>

```
// Custom Vertex Shader Jitter Injection (THREE.Material.onBeforeCompile)
uniform float uJitterLevel; // Defaults to 240.0 for authentic PS1 grid fidelity
varying vec4 vHPosition;

void main() {
    vec4 mvPosition = modelViewMatrix * vec4(position, 1.0);
```

```

gl_Position = projectionMatrix * mvPosition;

// Snapping vertices in clip space to simulate low-precision fixed-point calculations
vec4 snappedPos = gl_Position;
if (snappedPos.w != 0.0) {
    snappedPos.xyz /= snappedPos.w;
    snappedPos.xy = floor(snappedPos.xy * uJitterLevel + 0.5) / uJitterLevel;
    snappedPos.xyz *= snappedPos.w;
}
gl_Position = snappedPos;
vHPosition = gl_Position;
}

```

```

// Custom Fragment Shader for Ordered Bayer Dithering and Desaturation
uniform sampler2D tDiffuse;
uniform float uPixelSize;
uniform vec2 uResolution;
varying vec2 vUv;

float getBayerValue(vec2 pos) {
    int x = int(mod(pos.x, 4.0));
    int y = int(mod(pos.y, 4.0));
    int index = x + y * 4;

    float bayer;
    bayer = 0.0/16.0; bayer = 8.0/16.0; bayer = 2.0/16.0; bayer = 10.0/16.0;
    bayer = 12.0/16.0; bayer = 4.0/16.0; bayer = 14.0/16.0; bayer = 6.0/16.0;
    bayer = 3.0/16.0; bayer = 11.0/16.0; bayer = 1.0/16.0; bayer = 9.0/16.0;
    bayer = 15.0/16.0; bayer = 7.0/16.0; bayer = 13.0/16.0; bayer = 5.0/16.0;

    for(int i = 0; i < 16; i++) {
        if(i == index) return bayer[i];
    }
    return 0.0;
}

```

```

void main() {
    // Pixelation calculation
    vec2 dTexCoord = floor(vUv * uResolution / uPixelSize) * uPixelSize / uResolution;
    vec4 color = texture2D(tDiffuse, dTexCoord);

    // Apply 4x4 Bayer Ordered Dithering
    float luma = dot(color.rgb, vec3(0.299, 0.587, 0.114));
    vec2 screenPos = gl_FragCoord.xy / uPixelSize;
    float bayerValue = getBayerValue(screenPos);

    // Map output to pale, desaturated retro tones
    vec3 ditheredColor = color.rgb + (bayerValue - 0.5) * 0.15;
    ditheredColor = mix(ditheredColor, vec3(luma), 0.1); // Desaturate slightly
    gl_FragColor = vec4(ditheredColor * 0.85, 1.0); // Mimic darker CRT outputs
}

```

The game's visual design is closely modeled on the original art style, using simple geometric primitives and hand-drawn low-resolution textures :

- **Destructo Units:** Modeled as simple capsule geometries (THREE.CapsuleGeometry) with flat-shaded materials, avoiding smooth normals to preserve their blocky, pill-shaped profiles.<sup>4</sup> Hands are rendered as separate, detached floating spheres that transform relative to the body without a complex skeletal deformation rig.<sup>4</sup> The faces are mapped with simple  $64 \times 64$  texture atlases depicting distinct rectangular white eyes with black pupils and expressive cartoon mouths.
- **Base Station Factories:** Modeled as blocky, metal-riveted structures with blue or yellow accents and distinctive oversized brass keyhole details on the side panels, directly referencing the iconic team bases.
- **Environment:** Built from high-contrast, angular ground meshes featuring bright, high-saturation green grassy textures, jagged grey rock borders, and pinkish-purple cobblestone paths.<sup>8</sup> Sandbags are modeled as low-poly cylinders stacked in horizontal curves to form defensive bunkers. Cave entryways feature high-contrast, mossy stone patterns embedded with glowing blue mineral meshes.
- **Props:** Includes wooden target arrow signs, danger skull-and-crossbones signs, and dark metal trash cans overflowing with faceted white paper debris.<sup>10</sup> Flagpoles display simple, primary-colored flags matching the active team's color.
- **HUD:** A clean, screen-space 2D overlay. The top-left corner features stack depth bucket indicators, while the top-right displays the active squad queue with blue pill-shaped icons. The bottom corners show circular or heart-shaped health meters and ammunition gauges.

## Custom Physics Engine

The physics engine is a lightweight, deterministic rigid-body solver designed to handle

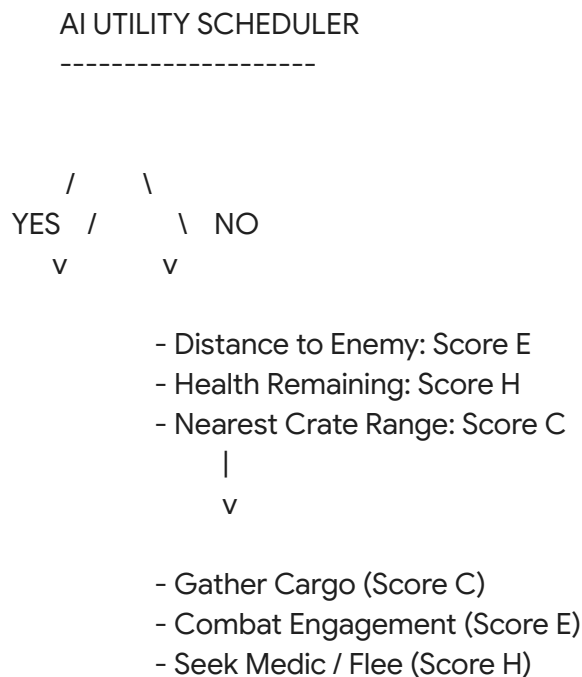
platforming, projectile trajectories, and high-impact knockback dynamics.<sup>4</sup> To avoid the high compute overhead of full convex-mesh collisions, the engine utilizes a Swept Axis-Aligned Bounding Box (AABB) model for entities, paired with signed distance fields (SDF) to calculate terrain heights.

To capture the elastic, bouncy movement profile of the original game, character movement transitions between four distinct physical states:

1. **Controlled Ground State:** Features high static friction, immediate acceleration, and zero inertia sliding, allowing for crisp, responsive player control.
2. **Airborne State:** Modeled with zero friction, a standard parabolic trajectory, and horizontal air control.
3. **Tumble State:** Triggered by high-impact attacks, enabling dynamic kinetic friction and elastic terrain bounces.<sup>4</sup>
4. **Bounce State:** Applied when an entity collides with terrain during a tumble, decaying kinetic energy and transitioning the entity into a recovery stun period.

## Custom AI Engine

The AI subsystem operates on a deterministic finite-state machine (FSM) combined with utility-based decision scoring, optimizing tactical operations for non-player-controlled units.<sup>10</sup> Each AI agent evaluates a tick-based scoring matrix to prioritize actions:



- **Gathering State:** Paths to the nearest unallocated crate drop using a high-density Navigation Mesh (NavMesh) built with dynamic obstacle avoidance.<sup>4</sup>
- **Assembly State:** Calculates the team's current tactical state and stacks crates on the base pad based on target builds requested via the L2 order matrix.<sup>4</sup>
- **Combat State:** Automatically targets hostile factions using line-of-sight raycasting. Includes flanking and strafing maneuvers to circle enemies while maintaining optimal firing distances.<sup>10</sup>
- **Support/Flee State:** Scans for allied medics or runs to defensive cover when HP drops below 25%.

## Custom Map and Level Chunking Engine

The level-rendering system relies on a quadtree-chunked terrain mesh with vertex colors to match the simple, textured styling of the original game.<sup>8</sup>

- **Destructible Assets:** Props such as trash cans, trees, and small buildings are mapped to a voxel-based deformation system. When their structural HP reaches zero, they fracture into discrete, physical low-poly debris components via an instanced mesh system, then dissolve into the terrain.
- **Water Shader:** Simulates water bodies using vertex displacement on a plane mesh. The vertex heights are animated using a sum of Gerstner waves to create stylized, faceted liquid surfaces that interact dynamically with entering entities (triggering splash particles and dynamic buoyancy physics).

## Custom Destructo Construction Engine

*Destructo* characters are assembled hierarchically using a central capsule mesh coupled with independent floating hand and boot meshes.<sup>4</sup> The engine updates these components using procedural animation formulas that calculate positions based on the character's movement velocity and state:

```
// Procedural Hover/Walk Bobbing Animation Equation
const bobbingFactor = Math.sin(clock.getElapsedTime() * walkFrequency) * walkAmplitude;
destructo.body.position.y = groundHeight + baseOffset + bobbingFactor;
```

```
// Floating hand tracking
destructo.leftHand.position.lerp(
  SCRATCH_VEC_A.set(bodyPos.x - offset, bodyPos.y + bobOffset, bodyPos.z +
    weaponOffset),
  0.3
```

);

## Custom Weapon System and Projectile Engine

The weapon system in *Destructo* replicates the low-poly aesthetic of the original game's arsenal, utilizing vibrant colors and visual directional indicators to map projectile vectors.

### WEAPON SCHEMATIC (LOW-POLY STYLE)

```
-----  
  
      /|  
===== | <-- Directional Vector Arrow  
      \| (Solid Red/Orange)  
[=====]  
| O | <-- Faceted Recoil Assembly  
|_ _|  
  ||  
  || <-- Blocky Hand Grip
```

The Projectile Engine manages bullet trajectories, rocket physics, and collision checking, handling hundreds of simultaneous active projectiles while maintaining 60+ FPS performance.

- **Raycast Projectiles (Instant Hit):** Bullet types (Pistol, SMG, Sniper Rifle) utilize Three.js Raycaster paths combined with a fast Octree spatial lookup to register immediate, frame-accurate target impacts.
- **Ballistic Projectiles (Physically Simulated):** Grenades, mortar shells, and rockets are registered as dynamic physical spheres within the Custom Physics Engine.<sup>4</sup> Their movement is updated each frame using explicit Euler integration:

$$x_{t+1} = x_t + v_t \cdot \Delta t$$

$$v_{t+1} = v_t + (g - C_{drag} \cdot v_t \cdot |v_t|) \cdot \Delta t$$

where  $g = [0, -9.81, 0]$  and  $C_{drag}$  represents the specific air resistance of the projectile.

## Custom Gear and Utility Subsystem

Provides passive buffs and active movement upgrades:

- **Jetpacks:** Consumes mana over time to apply a constant vertical force, enabling temporary flight and long-distance travel.
- **Magnetic Gloves:** Extends the pickup radius for standard cargo crates, letting units grab blocks from up to 3 units away.
- **Ballistic Shield Plates:** Attaches to the rear of the character model, automatically reducing damage taken from behind by 40%.

## Custom Particle and Special Effects Engine

The engine features a dedicated, low-overhead GPU-instanced Particle System designed to render explosive cartoon effects at a locked 60+ FPS:

- **Voxel Exploded Fragments:** Explosions burst into hundreds of small, spinning cubic colored chunks that bounce off the terrain before dissolving.
- **Bill-Boarded Smoke Rings:** Uses 2D low-res textures that rotate, scale up, and fade out to create distinct retro-styled fire and smoke effects.

## Custom Water and Buoyant Physics Engine

Water planes are simulated as a dynamic spatial boundary. When any entity enters water, the physics solver replaces standard gravity with a buoyant force  $F_b$  based on the volume of the object submerged:

$$F_b = \rho_{water} \cdot V_{submerged} \cdot g$$

Units caught in deep water without floating vehicles slowly consume oxygen (depleting HP) and experience 50% movement drag.

## Custom Vehicle Engine

Vehicles are modeled with unique physical profiles, steering mechanisms, and armor stats :

- **Motorcycles:** High-speed, low-turning-radius units with lightweight armor.
- **Tanks:** Built from 8 crates on a pad, featuring a rotating turret (R1 to re-center) and high armor resistance.<sup>4</sup>
- **Helicopters / Gunships:** True 3D flight capabilities with lock-on rocket pods.
- **Combat Mechs:** Heavy bipedal walkers that can walk through shallow water, crush small structures, and tow heavy Mega Crates.<sup>4</sup>

## Custom Base and Team Management Engine

Manages team status, base pad coordinates, player allocations, and core structure states. If a base's command factory is destroyed, its D-Builder pad deactivates, locking that team out of future spawns.

## Custom Neutrals and Wildlife Engine

Handles the behavior paths of native map animals:

- **Stray Sheep / Cows:** Wander randomly and can be ridden or packed with remote explosives.

- **Feral Wolves:** Guard central choke points and attack any nearby units, regardless of team color.
- **Crate Slimes:** Rare slimes that eat loose crates on the map; defeating them releases high-tier Super Charged crates.<sup>4</sup>

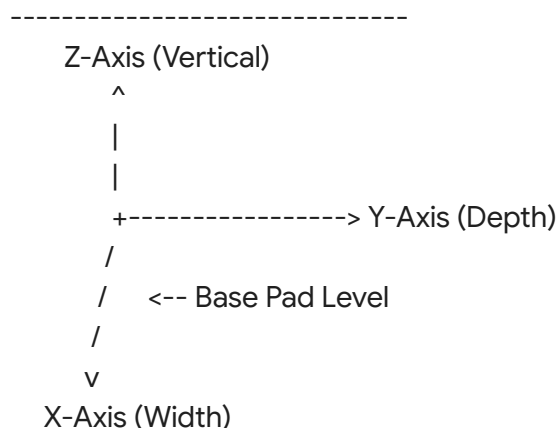
## Custom Structures and Environment Engine

Governs sandbag barriers, cave tunnels, and signposts. Sandbags act as physical walls that degrade under heavy shell fire, while mineral caves act as capture points, spawning high-tier resources when held.

## The D-Builder Crate Stacking System and Recipe Matrix

The core gameplay loop of *Destructo* centers around the dynamic crate collection, stacking, and construction system, known as the **D-Builder**.<sup>2</sup> Each team base contains a dedicated  $2 \times 2 \times$  coordinate grid layout pad.<sup>2</sup> Stacking physical crates within this three-dimensional spatial matrix determines the exact unit, weapon, or vehicle manufactured once the stack is broken open.<sup>2</sup>

### D-BUILDER PAD COORDINATE SCHEME



The stack is processed as a binary 3D spatial occupancy matrix where the index  $[x, y, z]$  is occupied (1) or vacant (0) on the  $2 \times 2 \times$  grid:

```
// Example D-Builder Stacking Pad Structure
const padOccupancy = [ , ], // Z = 0 (Base level: [[x0y0, x1y0], [x0y1, x1y1]])
  [ , ] // Z = 1 (Upper level)
];
```

The physical combinations of crates determine the object spawned upon destruction <sup>2</sup>:

| Stacking Configuration                  | Spatial Pattern                      | Standard Output                          | Super Charged Output                        |
|-----------------------------------------|--------------------------------------|------------------------------------------|---------------------------------------------|
| <b>1 Crate</b>                          | \$ = 1\$                             | Light Weapon (Pistol)                    | Super Laser Pistol (Stuns and burns)        |
| <b>2 Crates Horizontal</b> <sup>4</sup> | \$ = 1, = 1\$                        | Grenade Packs (3x standard) <sup>4</sup> | Cluster Bomb Packs (Splits on blast)        |
| <b>2 Crates Vertical</b>                | \$ = 1, = 1\$                        | Scout Destructo                          | Super Scout (Shielded & extremely fast)     |
| <b>4 Crates Horizontal</b> <sup>4</sup> | Row <b>Z</b> = completely occupied   | Heavy Weapon (Shotgun) <sup>4</sup>      | Super Shotgun (High-impact pulse fire)      |
| <b>4 Crates Square</b> <sup>2</sup>     | Plane <b>Z</b> = completely occupied | Commando Destructo                       | Super Commando (Heavily armored, dual guns) |
| <b>4 Crates Vertical</b> <sup>4</sup>   | Two parallel vertical columns of 2   | Medic Destructo <sup>4</sup>             | Super Medic (Aura heal, revive capacity)    |
| <b>8 Crates Cube</b> <sup>4</sup>       | Full occupancy of                    | Standard Tank <sup>4</sup>               | Hover Tank                                  |

|  |                              |  |                                 |
|--|------------------------------|--|---------------------------------|
|  | the $2 \times 2 \times$ grid |  | (Shielded, fires plasma bursts) |
|--|------------------------------|--|---------------------------------|

Crates placed on a pad automatically snap and fuse into a single composite mesh, which cannot be picked up or moved as separate blocks. To split a fused crate, players can utilize a Ninja's katana sweep or a deconstruction tool to break the connections and return the crates to their individual physical states.

## Weapon-Specific Physics and Knockback Dynamics

The knockback calculations are governed by the application of an instantaneous impulse vector ( $I$ ) to a Destructo entity of mass  $m$ .<sup>4</sup> The physics tick computes the new velocity vector  $v_{new}$  using:

$$v_{new} = v_{old} + \frac{I}{m}$$

Once the impulse is applied, the entity enters the **Tumble State**. During this state, horizontal sliding velocity decays under a custom dynamic friction model, while vertical velocity is subject to a gravity scalar  $g_{scalar}$  and a high air-drag coefficient  $C_d$ . The state updates as follows:

$$x_{t+1} = x_t + v_t \cdot \Delta t$$

$$v_{t+1} = (v_t - \hat{v}_t \cdot \mu_k \cdot g_{scalar} \cdot \Delta t) - (C_d \cdot v_t \cdot |vecv_t| \cdot \Delta t)$$

where  $\hat{v}_t$  is the normalized horizontal velocity vector and  $\mu_k$  is the dynamic kinetic friction of the surface.

### KNOCKBACK STATE FLOW

-----

```
|
| <-- Weapon Impact Applied:
|   Vector I (Force/Mass)
```

```

v
|
| <-- Bounds check with ground:
|   SDF Collision
v
|
| <-- Dynamic Friction decay:
|   Velocity falls below V_threshold
v
|
| <-- Recovery Timer expires
v

```

To precisely recreate the weapon-specific physics quirks of *Team Buddies*, the impact vectors and state flags are meticulously configured per weapon type <sup>4</sup>:

| Weapon Class       | Base Damage | Projectile Velocity | Base Knockback Vector (I) | Induced Physical State | Trigger Mechanics & Stun Duration              |
|--------------------|-------------|---------------------|---------------------------|------------------------|------------------------------------------------|
| Standard Punch     | 10          | Immediate           | [4.5, 1.5, 4.5]           | Stagger State          | Triggers a 0.3s movement lock. <sup>4</sup>    |
| Heavy Charge Punch | 30          | Immediate           | [12.0, 6.0, 12.0]         | Tumble State           | Forces held items to drop; 0.8s recovery stun. |
| Cyborg Heavy Slam  | 45          | Immediate           | [18.0, 9.5, 18.0]         | Tumble State           | Radial ground smash; causes 1.2s recovery      |

|                                    |    |           |                 |               |                                                                   |
|------------------------------------|----|-----------|-----------------|---------------|-------------------------------------------------------------------|
|                                    |    |           |                 |               | stun. <sup>4</sup>                                                |
| <b>Pistol</b>                      | 8  | 120.0 m/s | [0.8, 0.0, 0.8] | Stagger State | Interrupts firing animations with no knockdown.                   |
| <b>Assault Rifle</b> <sup>17</sup> | 15 | 180.0 m/s | [2.2, 0.2, 2.2] | Slide State   | Causes targets to slide backward along the terrain. <sup>17</sup> |
| <b>Shotgun (Point Blank)</b>       | 60 | 90.0 m/s  | [18.0, 5.0, 1]  | Tumble State  | High-impact close-range blowback. <sup>4</sup>                    |
| <b>Shotgun (Max Range)</b>         | 10 | 90.0 m/s  | [2.0, 0.1, 2.0] | Stagger State | Minimal pushback at maximum distances.                            |
| <b>Heavy Machine Gun</b>           | 12 | 150.0 m/s | [1.5, 0.1, 1.5] | Slide State   | Continuous rapid-fire impacts lock the target in place.           |
| <b>Cannon Shell</b>                | 80 | 60.0 m/s  | [25.0, 12.0, 1] | Tumble State  | Launches targets in high parabolic arcs; 1.5s stun. <sup>4</sup>  |
| <b>Grenades /</b>                  | 70 | 35.0 m/s  | Radial Blast    | Tumble        | Throws                                                            |

|                |    |          |                     |              |                                                     |
|----------------|----|----------|---------------------|--------------|-----------------------------------------------------|
| Mines          |    |          | Vector              | State        | targets outward from the blast center. <sup>4</sup> |
| Rocket Missile | 90 | 80.0 m/s | Radial Blast Vector | Tumble State | High-speed impact followed by a radial explosion.   |

The First Dimension: Military-Themed Destructos

The First Dimension of *Destructo* is styled around a high-impact, satirical military theme. The standard rounded, pill-shaped Destructos face off in colored divisions (Blue, Red, Yellow, Green, Pink, Purple, Orange, and Brown).<sup>4</sup> Each unit displays flat, billboarded, screen-space 2D health and mana bars above their head, keeping tactical indicators highly visible during chaotic combat.

BILLBOARDED 2D SCREEN-SPACE OVERLAY

-----

[ 100 / 100 HP ] <-- Flat UI Bar (Green)  
[ 50 / 50 MP ] <-- Flat UI Bar (Blue)



The 10 Military-Themed Destructos of the First Dimension are specified below:

|        |      |      |      |         |    |        |       |
|--------|------|------|------|---------|----|--------|-------|
| Destru | Base | Base | Core | Special | MP | Cooldo | Skill |
|--------|------|------|------|---------|----|--------|-------|

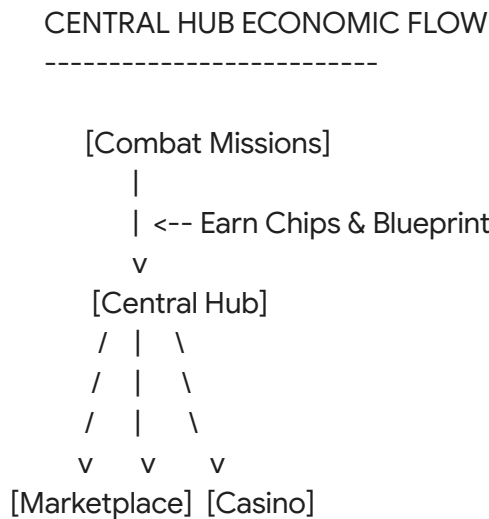
| cto<br>Class<br>Name           | HP  | MP  | Weapo<br>n<br>Special<br>ty | Skill<br>Name      | Cost | wn  | Code-L<br>evel<br>Mecha<br>nical<br>Effects                                                          |
|--------------------------------|-----|-----|-----------------------------|--------------------|------|-----|------------------------------------------------------------------------------------------------------|
| <b>Scout</b> <sup>4</sup>      | 100 | 50  | Combat<br>Pistol            | Tactical<br>Sprint | 15   | 8s  | Boosts<br>movem<br>ent<br>speed<br>by 60%<br>and<br>grants<br>immunit<br>y to<br>slow<br>effects.    |
| <b>Medic</b> <sup>4</sup>      | 120 | 100 | Needle<br>Dart<br>Gun       | Nanite<br>Mist     | 40   | 12s | Deploys<br>a<br>localize<br>d<br>healing<br>cloud<br>that<br>restore<br>s 15<br>HP/sec<br>to allies. |
| <b>Sniper</b><br><sub>17</sub> | 90  | 80  | High-C<br>aliber<br>Rifle   | Cloakin<br>g Grid  | 35   | 15s | Grants<br>invisibili<br>ty for<br>6s; next<br>shot<br>deals<br>2.5x<br>damag<br>e.                   |

|                                   |     |     |                        |                       |    |     |                                                                                                    |
|-----------------------------------|-----|-----|------------------------|-----------------------|----|-----|----------------------------------------------------------------------------------------------------|
| <b>Gunner</b>                     | 150 | 60  | Machin<br>egun         | Overdri<br>ve Fire    | 30 | 10s | Double<br>s firing<br>velocity<br>and<br>reload<br>speed<br>while<br>stationa<br>ry for<br>4s.     |
| <b>Explosi<br/>ves<br/>Master</b> | 130 | 70  | Mortar<br>Grenad<br>es | Cluster<br>Toss       | 45 | 14s | Throws<br>an<br>explosiv<br>e that<br>splits<br>into 5<br>sub-gre<br>nades<br>upon<br>impact.      |
| <b>Comm<br/>ando</b>              | 160 | 50  | Dual<br>Shotgu<br>ns   | Shockw<br>ave<br>Jump | 25 | 6s  | Leap-sl<br>ams<br>into a<br>location<br>,<br>dealing<br>radial<br>damag<br>e and<br>knockb<br>ack. |
| <b>Officer</b>                    | 140 | 100 | Officer<br>Carbine     | Rallying<br>Cry       | 50 | 18s | Boosts<br>nearby<br>friendly<br>unit<br>movem<br>ent and<br>attack                                 |

|                          |     |    |                 |                 |    |     |                                                                        |
|--------------------------|-----|----|-----------------|-----------------|----|-----|------------------------------------------------------------------------|
|                          |     |    |                 |                 |    |     | by 25%<br>for 8s.                                                      |
| <b>Saboteur</b>          | 110 | 90 | Proximity Mines | EMP Plant       | 35 | 11s | Places a trap that drains target mana and disables vehicles.           |
| <b>Heavy Shield</b>      | 220 | 40 | Shield / Pistol | Phalanx Barrier | 20 | 9s  | Deploys a front-facing energy shield that blocks incoming projectiles. |
| <b>Tactical Engineer</b> | 130 | 80 | SMG / Wrench    | Sentry Deploy   | 60 | 20s | Spawns a machine gun turret targeting enemies within 15 units.         |

## Economic Architecture: Shop, Marketplace, and Casino

*Destructo* extends the classic tactical formula of *Team Buddies* by introducing a persistent economic lobby system. Between combat missions, players return to a central military hub to spend their hard-earned game chips on gear, blueprints, and upgrades.<sup>18</sup>



## The Marketplace

The Marketplace is the core progression shop where players unlock new tactical tools for their army.<sup>18</sup> Transactions are processed using chips, the main currency earned by completing primary objectives and secondary mission challenges.<sup>13</sup>

- **Weapon Blueprint Licensing:** Players purchase manufacturing blueprints to add new weapon combinations to their D-Builder pads during battle (e.g., spending 5,000 chips to unlock the Flamethrower recipe).<sup>13</sup>
- **Gear & Utility Items:** Consumable items can be purchased and equipped to starting units, such as rocket-propelled jump packs, passive armor plating, and magnetic crate-carrying gloves.
- **Stat Upgrades:** Permanent training upgrades for *Destructo* types (e.g., adding +10% max HP or accelerating mana recovery rates).

## The Casino

The Casino offers minigames and gambling activities where players can bet their chips for a chance to win exclusive cosmetic skins, prototype blueprints, or jackpots.<sup>13</sup>

- **The Crate Spinner:** A classic slot machine styled after the D-Builder crate pad. Matching three Super Charged crates payouts rare structural skins and cosmetic hats.<sup>4</sup>
- **Double-or-Nothing Card Game:** A high-low card minigame where players bet their

mission earnings against a dealer.

- **The Blueprint Raffle:** A ticketing system where players spend excess chips for a chance to win experimental prototype weaponry.

### The D-Builder Studio

The custom cosmetic garage where players edit their squad uniforms, modify victory emotes, and manage active weapon loadouts for future missions.

## Third-Party Library Integration and Optimization Matrix

To build a feature-rich, high-performance web-native game, *Destructo* integrates 10 specialized third-party libraries.<sup>5</sup> These tools handle complex systems like spatial math, audio, and physics, allowing the custom core engines to run with maximum efficiency.

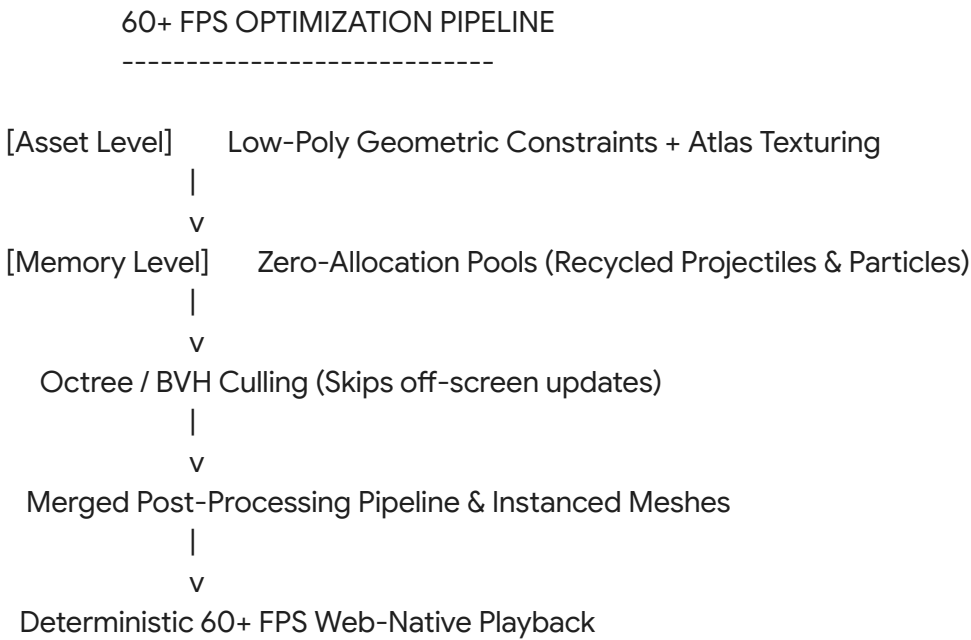
| Library Name     | Exact Purpose in Engine Suite                                                          | Core Integration API Calls & Implementation Hooks                           | Performance Impact & Overhead Mitigation                                                                 |
|------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| cannon-es        | Back-end physics calculations for rigid bodies, friction, and collisions. <sup>5</sup> | CANNON.World(), Body.applyImpulse(), allowSleep: true, matching transforms. | Resolves complex box and sphere collision checks using simple shape math, reducing overhead.             |
| GSAP (GreenSock) | Smooth menu transitions, UI sliding, and character animation tweening. <sup>19</sup>   | gsap.to(), custom canvas render updates, CPU property mapping.              | Replaces expensive CSS layout calculations with lightweight, frame-synced transformations. <sup>19</sup> |
| Howler.js        | Spatial, 3D panning audio engine for gunshots, voice lines, and explosions.            | Howl(), web audio node setups, spatial listener tracking.                   | Keeps audio processing off the main rendering thread, avoiding sound lag and frame stutters.             |

|                          |                                                                    |                                                                    |                                                                                               |
|--------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>postprocessing</b>    | High-performance render pass chain for pixelation and LUT effects. | EffectComposer, RenderPass, custom ShaderPass setups.              | Combines multiple post-processing effects into a single GPU pass to save memory.              |
| <b>three-pathfinding</b> | NavMesh generation and pathfinding operations for tactical AI.     | Pathfinding.createZone(), NavMesh boundaries, agent steering.      | Avoids heavy point-to-point path recalculations, processing spatial queries in under 0.5ms.   |
| <b>three-mesh-bvh</b>    | Accelerates raycasting queries for fast weapon checks.             | Mesh.geometry.computeBoundsTree(), raycasting acceleration.        | Accelerates line-of-sight checks by up to 100x, allowing hundreds of bullet checks per frame. |
| <b>Stats.js</b>          | Real-time performance monitoring of FPS and MS draw cycles.        | stats.begin(), page container setups, requestAnimationFrame ticks. | Provides real-time rendering stats with minimal impact on performance.                        |
| <b>lil-gui</b>           | In-engine developer debugging console and variable adjuster.       | GUI.add(), parameter tracking folders, physics variables.          | Offers a lightweight, zero-overhead diagnostic control overlay for debugging during gameplay. |
| <b>Zdog</b>              | Handles rendering for lightweight, flat 2D pseudo-3D UI overlays.  | Vector-based SVG structures, mathematical flat models.             | Renders lightweight, vector-based HUD symbols without requiring full 3D rendering             |

|          |                                                        |                                                       |                                                                                |
|----------|--------------------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------|
|          |                                                        |                                                       | overhead.                                                                      |
| NippleJS | Virtual on-screen joysticks for intuitive mobile play. | nipplejs.create(), touch coordinates, dual joysticks. | Offloads touch and mouse events to efficient, non-blocking hardware listeners. |

## Engine Optimization and Deployment Strategy

To ensure *Destructo* maintains a locked framerate of 60+ FPS on a wide range of devices (including older mobile phones and low-end laptops), the engine utilizes advanced optimization techniques across all subsystems.



### Advanced Memory Management

A common bottleneck in browser-based games is garbage collection (GC) stutter, caused by creating and destroying temporary objects like projectiles, particles, and vectors during gameplay. *Destructo* addresses this with a zero-allocation architecture:

- **Object Pooling:** Projectiles, particle groups, and floating UI indicators are created at launch in fixed-size arrays. When a rocket is fired, an inactive projectile is pulled from the

pool, initialized with active values, and returned to the pool once it detonates. This completely avoids allocating memory on the fly during action sequences.

- **Vector Re-use:** Instead of creating new Vector3 objects for physics calculations, the engines reuse pre-allocated global scratch variables:

```
// Preventing GC Stutter through Global Vector Re-use
const SCRATCH_VEC_A = new THREE.Vector3();
const SCRATCH_VEC_B = new THREE.Vector3();

function calculateVelocity                    {
    // Direct modification of global scratch vectors avoids allocating new memory
    SCRATCH_VEC_A.copy(targetPos).sub(entity.position).normalize();
    SCRATCH_VEC_B.copy(entity.velocity).addScaledVector(SCRATCH_VEC_A, force);
    entity.velocity.copy(SCRATCH_VEC_B);
}
```

## Advanced Rendering Optimizations

- **Instanced Mesh Rendering:** Static environmental props, trees, and grass are merged and rendered using Three.js InstancedMesh. This allows the engine to draw thousands of detailed meshes in a single GPU draw call.
- **Frustum and Occlusion Culling:** Entities and structural elements outside the camera's view frustum are skipped during rendering passes. AI updates for off-screen units are throttled, running at lower update rates depending on their distance from the player.
- **Texture Atlasing:** Rather than loading separate image files for each entity and weapon, all textures are compiled into a single high-density  $2048 \times 2048$  texture atlas. This minimizes texture swaps on the GPU, streamlining the rendering process and keeping the gameplay buttery smooth.

## Works cited

1. Team Buddies - PlayStation 1 | Midway Games - GameStop, accessed on July 10, 2026,  
<https://www.gamestop.com/video-games/retro-gaming/products/team-buddies--playstation-1/10131082.html>
2. Beta + Cancelled third person shooter Video Games - Page 9 of 49 - Unseen64, accessed on July 10, 2026,

- <https://www.unseen64.net/tag/third-person-shooter/page/9/>
3. glazball's game collection (6/24/25) [Platform: Sony PlayStation], accessed on July 10, 2026, <http://www.droog.x10host.com/page33.html>
  4. Team Buddies - Grokipedia, accessed on July 10, 2026, [https://grokipedia.com/page/Team\\_Buddies](https://grokipedia.com/page/Team_Buddies)
  5. How to Create a PS1-Inspired Jitter Shader with React-Three-Fiber - Codrops, accessed on July 10, 2026, <https://tympanus.net/codrops/2024/09/03/how-to-create-a-ps1-inspired-jitter-shader-with-react-three-fiber/>
  6. Three.js Visual & Interactive Encyclopedia - A Complete Guide, accessed on July 10, 2026, <https://neuralpixelgames.github.io/threejs-visual-guide/>
  7. Three.js retro arcade effect using post-processing - AppFoundry, accessed on July 10, 2026, <https://www.appfoundry.be/insights/three-js-retro-arcade-effect-using-post-processing>
  8. Team Buddies - Wikipedia, accessed on July 10, 2026, [https://en.wikipedia.org/wiki/Team\\_Buddies](https://en.wikipedia.org/wiki/Team_Buddies)
  9. Building a Real-Time Dithering Shader | daily.dev, accessed on July 10, 2026, <https://daily.dev/posts/building-a-real-time-dithering-shader-aexr8ikzk>
  10. Team Buddies (USA), accessed on July 10, 2026, [https://www.videogamemanual.com/ps1/Team%20Buddies%20\(USA\).pdf](https://www.videogamemanual.com/ps1/Team%20Buddies%20(USA).pdf)
  11. Team buddies different ranks by RagingBullet on DeviantArt, accessed on July 10, 2026, <https://www.deviantart.com/ragingbullet/art/Team-buddies-different-ranks-852771417>
  12. Crate Expectations - All The Tropes, accessed on July 10, 2026, [https://allthetropes.org/wiki/Crate\\_Expectations](https://allthetropes.org/wiki/Crate_Expectations)
  13. Team Buddies - Guide and Walkthrough - PlayStation - By BombZero - GameFAQs, accessed on July 10, 2026, <https://gamefaqs.gamespot.com/ps/374591-team-buddies/faqs/11935>
  14. Graphics Programming Weekly - Database - Jendrik Illner, accessed on July 10, 2026, [https://www.jendrikillner.com/article\\_database/](https://www.jendrikillner.com/article_database/)
  15. Page 2 | Retrospective: Team Buddies | Eurogamer.net, accessed on July 10, 2026, <https://www.eurogamer.net/team-buddies-retrospective?page=2>
  16. N3 tactical guide to Hassassin Bahram, the Haqqislamite Military Intelligence Service, accessed on July 10, 2026, <https://forum.corvusbelli.com/threads/n3-tactical-guide-to-hassassin-bahram-the-haqqislamite-military-intelligence-service.571/>
  17. What Are You Playing 8 (2023 Reboot) - Giant in the Playground Forums, accessed on July 10, 2026, <https://forums.giantitp.com/showthread.php?p=25765072>
  18. zhoufangqin/AwesomeUnityCommunity: A curated list of awesome Unity assets, resources, and more. - GitHub, accessed on July 10, 2026, <https://github.com/zhoufangqin/AwesomeUnityCommunity>